Lab 7: Fuzzing for Input Validation Bugs

Aditya Vats | Repo: L6_L7_31_coupon_validator (lab7) | Python 3.14, Hypothesis 6.170, pytest 9.2

The handout's `processor.py` and `test_processor.py` are `coupon_validator.py` and `test_coupon_validator.py` in the assigned repo. Screenshots below are captured terminal output from the actual runs.

Step 1: Install dependencies

`pip install hypothesis` and `pip install pytest`

Step 2: Run the buggy application

```
$ python coupon_validator.py
==== Running split_code ====
Enter a coupon code (e.g. save-20): save-20
Prefix and Value: ('SAVE', 20)

==== Running most_expensive ====
Enter item prices as CSV (e.g. 199,49.5,320): 199,49.5,320
Most Expensive Item: 320.0

==== Running expiry_days ====
Enter an expiry date as YYYY-MM-DD: 2025-03-15
Days Since 2024: 440
```

Buggy code with valid input: works as expected.

```
$ python coupon_validator.py
==== Running split_code ====
Enter a coupon code (e.g. save-20): save20
Traceback (most recent call last):
  File "/home/av8/Documents/SE/SE-Labs/Lab-07/coupon_validator.py", line 30, in <module>
    main()
    ~~~~^^
  File "/home/av8/Documents/SE/SE-Labs/Lab-07/coupon_validator.py", line 16, in main
    result = split_code(user_input)
  File "/home/av8/Documents/SE/SE-Labs/Lab-07/coupon_validator.py", line 2, in split_code
    prefix, value = code.strip().upper().split('-')
    ^^^^^^^^^^^^^
ValueError: not enough values to unpack (expected 2, got 1)
```

Buggy code with a code that has no hyphen: crashes with an unhandled ValueError.

Step 3: Complete the tests using Hypothesis

The three starter `no_crash` tests were completed using the handout's pattern, feeding each function random text and None. Nine more property tests were added, because a no-crash test cannot detect wrong answers (for example, returning NaN) and purely random text rarely gets close to a valid format. They cover non-text types, near-valid inputs, and correct results on valid input.

Step 4: Run the tests on the buggy code

```
$ pytest test_coupon_validator.py
========================================= test session starts ==========================================
platform linux -- Python 3.14.0, pytest-9.2.0, pluggy-1.7.0
rootdir: /home/av8/Documents/SE/SE-Labs/Lab-07
plugins: hypothesis-6.170.14
collected 12 items

test_coupon_validator.py FFFFFFF.F.F.                                                    [100%]

[... 362 lines of Hypothesis tracebacks omitted to fit the page ...]

====================================== short test summary info =======================================
FAILED test_coupon_validator.py::test_split_code_no_crash - ExceptionGroup: Hypothesis found 3 distinct fail
FAILED test_coupon_validator.py::test_most_expensive_no_crash - ExceptionGroup: Hypothesis found 2 distinct
FAILED test_coupon_validator.py::test_expiry_days_no_crash - ExceptionGroup: Hypothesis found 2 distinct fai
FAILED test_coupon_validator.py::test_non_text_input_is_rejected[split_code] - ExceptionGroup: Hypothesis fo
FAILED test_coupon_validator.py::test_non_text_input_is_rejected[most_expensive] - ExceptionGroup: Hypothesi
FAILED test_coupon_validator.py::test_non_text_input_is_rejected[expiry_days] - ExceptionGroup: Hypothesis f
FAILED test_coupon_validator.py::test_split_code_returns_clean_result_or_none - ValueError: Exceeds the limi
FAILED test_coupon_validator.py::test_most_expensive_rejects_any_bad_price - ExceptionGroup: Hypothesis foun
FAILED test_coupon_validator.py::test_expiry_days_only_accepts_real_dates - ExceptionGroup: Hypothesis found
==================================== 9 failed, 3 passed in 6.19s =====================================
```

Result: 9 failed, 3 passed. The 3 passes feed only valid input, which the buggy code already handled.

Bugs found

Function	Input	Buggy behaviour	Fixed behaviour
split_code	None, 0, b''	AttributeError / TypeError	Returns None
split_code	'' (no hyphen)	ValueError: not enough values to unpack	Returns None
split_code	'-'	ValueError: int('')	Returns None
split_code	'save-' + 5000 nines	ValueError: exceeds int digit limit	Returns None (value limited to 9 digits)
most_expensive	None, 0, b''	AttributeError / TypeError	Returns None
most_expensive	''	ValueError: could not convert to float	Returns None
most_expensive	'nan'	No crash: returns nan as the price	Returns None
expiry_days	None, 0, b''	AttributeError / TypeError	Returns None
expiry_days	''	ValueError: int('')	Returns None
expiry_days	'0000-0-0'	No crash: returns -738790 for an impossible date	Returns None

Step 5: Fix the application

Each function now checks the input type and matches a strict ASCII pattern before converting anything, and returns None if the input is invalid. `expiry_days` also checks that the date really exists, and `most_expensive` rejects non-finite prices. `main()` prints a message rather than crashing. Results for valid input are unchanged.

```
$ python coupon_validator.py
==== Running split_code ====
Enter a coupon code (e.g. save-20): save-20
Prefix and Value: ('SAVE', 20)

==== Running most_expensive ====
Enter item prices as CSV (e.g. 199,49.5,320): 199,49.5,320
Most Expensive Item: 320.0

==== Running expiry_days ====
Enter an expiry date as YYYY-MM-DD: 2025-03-15
Days Since 2024: 440
```

Fixed code with valid input: same output as Step 2.

```
$ python coupon_validator.py
==== Running split_code ====
Enter a coupon code (e.g. save-20): save20
Invalid coupon code

==== Running most_expensive ====
Enter item prices as CSV (e.g. 199,49.5,320): 199,,abc
Invalid price list

==== Running expiry_days ====
Enter an expiry date as YYYY-MM-DD: 2025-13-45
Invalid expiry date
```

Fixed code with invalid input at all three prompts: clear messages and no crash.

Step 6: Re-run the tests on the fixed code

```
$ pytest test_coupon_validator.py
========================================= test session starts ==========================================
platform linux -- Python 3.14.0, pytest-9.2.0, pluggy-1.7.0
rootdir: /home/av8/Documents/SE/SE-Labs/Lab-07
plugins: hypothesis-6.170.14
collected 12 items

test_coupon_validator.py ............                                                          [100%]

========================================== 12 passed in 2.25s ==========================================
```

Result: 12 passed.